Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 6 — Solutions

Lists and nested lists

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — The list type

A1 *Follow the list*

	line	t afterwards
	`t = [5, 3, 8]`	`[5, 3, 8]`
1	`t.append(3)`	`[5, 3, 8, 3]`
2	`t.insert(0, 9)`	`[9, 5, 3, 8, 3]`
3	`x = t.pop(0)`	`[5, 3, 8, 3]` and `x` is `9`
4	`t.remove(3)`	`[5, 8, 3]` — only the *first* 3 goes
5	`y = t.pop(1)`	`[5, 3]` and `y` is `8`
6	`t.append(x + y)`	`[5, 3, 17]`

The two traps are lines 4 and 5. `remove` takes out one occurrence, the first, and leaves the other 3 where it was. `pop(1)` takes the item at **position** 1 — which after line 4 is the 8, not the 5.

`t.remove(4)` raises `ValueError: list.remove(x): x not in list`. There is no 4 to remove, and `remove` does not quietly do nothing.

A2 *Which lines lose the list?*

a) Lines **1, 4 and 6** leave `g` as `None`.

b)

	line	g afterwards
2	`g = sorted(g)`	`[1, 2, 3]` — a new list, sorted
3	`g.append(4)`	`[3, 1, 2, 4]` — the same list, changed
5	`g = g + [4]`	`[3, 1, 2, 4]` — a new list, built by `+`

c) A method that changes the list gives back `None`, so never assign its result; a function or operator that builds a new list gives it back, so you must.

Worth noticing. Put this next to sheet 5, B2. There the rule was "a string method never changes the string, so always catch what it gives back". Here it is the exact opposite. Neither rule is "always assign" or "never assign": the question is whether the operation changes the thing you have, or hands you a new one. Lines 3 and 5 end with the same list printed — and they are not the same, which is what A3 is about.

A3 *Draw it*

	prints	the picture at the end
(a)	`[1, 2, 3]`	one list, `a` and `b` both pointing at it
(b)	`[1, 2]`	two lists: `b` on `[1, 2]`, `a` on `[1, 2, 3]`
(c)	`[1, 2]`	two lists: `a` on `[1, 2]`, `b` on `[99, 2]`

The difference between (a) and (b), in one sentence: **`a.append(3)` changes the list that both names point at, while `a = a + [3]` builds a new list and moves only `a` to it.**

In (b), `b` is left pointing at the original list, which nobody changed. In (c), `a[:]` made a second list at the start, so the names never shared one.

Worth noticing. The test for any aliasing question: does the line *change a box*, or *move an arrow*? `append`, `sort`, `b[0] = 99` change a box — every name pointing at that box sees it. Any line of the form `name = ...` moves one arrow — nothing else is affected. (c) is the tricky one: `b[0] = 99` looks like an assignment, but the thing on the left is an *item* of a list, not a name, so it changes a box.

A4 *The copy that is not a copy*

a) It prints the curved grades twice:

```
before: [30, 30, 26, 17, 29]
after:  [30, 30, 26, 17, 29]
```

b) `curved = grades`. It does not copy anything: it attaches a second name to the same list, so every change made through `curved` is a change to `grades`.

c)

```
curved = grades[:]
```

`grades.copy()` and `list(grades)` work equally well.

Worth noticing. The comment the programmer would have written next to `curved = grades` is "keep a copy". That belief is the bug. It is also why the bug survives testing: if you only ever print `curved`, the program looks perfect.

A5 *First come, first served*

```
signups = ["Linus", "Ada", "Linus", "Grace", "Ada", "Alan", "Ada"]

# start with an empty list
# for every sign-up, add the name only if it is not there yet
# print the list, and how many sign-ups were left out

unique = []
for name in signups:
    if name not in unique:
        unique.append(name)

print(unique)                          # ['Linus', 'Ada', 'Grace', 'Alan']
print(len(signups) - len(unique))      # 3
```

It is the filter from the slide *Building a list*, with the test asked of the list being built instead of a fixed number.

d) `sorted(unique)` gives `["Ada", "Alan", "Grace", "Linus"]`. That puts Ada first, and Linus signed up before her. On a waiting list the order is the information, and sorting throws it away.

Worth noticing. Every `name not in unique` is a linear search through the list built so far. For a waiting list that costs nothing worth measuring. Next week brings a structure that removes duplicates in one call, and it keeps no order at all. This loop stays the way to do it when the order matters.

Part B — Iterating over lists

B1 *Six expressions on a table*

	expression	value
1	`t[0]`	`[3, 8, 1]` — a whole row
2	`t[2][0]`	`4`
3	`t[0][2]`	`1`
4	`t[-1][-1]`	`5` — the last item of the last row
5	`len(t[1])`	`3` — the number of columns
6	`t[1][1:]`	`[2, 9]` — a slice of row 1

`9` is `t[1][2]`. The middle item is `t[1][1]`, which is `2`.

Numbers 2 and 3 are the pair to compare: same two indices, opposite order, different answers. Row first, then column.

B2 *It tries, and it cannot*

a) `['ada', 'alan', 'grace']` — unchanged.

b) `name` is a name for the current item; `name = ...` moves that name to a new string and leaves the list alone. And there is no way to change the item through `name` anyway, because strings are immutable: `.title()` builds a new string, it cannot alter the old one.

c)

```
for i in range(len(names)):
    names[i] = names[i].title()
```

d)

```
names = [n.title() for n in names]
```

Worth noticing. (c) and (d) are different in a way that matters after this week. (c) changes the existing list: anyone else holding a name for it sees the capitals. (d) builds a new list and moves `names` to it: anyone else still has the old one. Usually either is fine. When it is not, you now know which to pick.

B3 *Comprehensions, both ways*

```
squares = [n * n for n in range(1, 6)]
short = [w for w in words if len(w) <= 3]
loud = [w.upper() for w in words]
```

They give `[1, 4, 9, 16, 25]`, then `['the', 'fox', 'the', 'dog']`, then all nine words in capitals.

And back:

```
lengths = []
for w in words:
    if w != "the":
        lengths.append(len(w))
```

It gives `[5, 5, 3, 5, 4, 4, 3]`: the lengths of every word except the two `the`s.

The translation is mechanical in both directions. The expression at the front of the comprehension is what goes inside `append`; the `for` is the loop; the `if`, if there is one, wraps the `append`.

B4 *A table of scores*

a) A row is a list, so `sum` works on it directly:

```
for row in scores:
    print(sum(row))          # 82, 70, 89, 66
```

b) A column is not a list, so walk it by position: the outer loop picks a column `c`, the inner loop goes down the rows.

```
n_rows = len(scores)
n_cols = len(scores[0])

for c in range(n_cols):
    total = 0
    for r in range(n_rows):
        total += scores[r][c]
    print(f"exam {c + 1}: {total / n_rows:.2f}")
```

The means are `24.25`, `25.00` and `27.50`. `total = 0` sits *inside* the outer loop: one total per column. Put it outside and the second exam's mean includes the first exam's marks.

c) Each column of `scores` becomes a row of `flipped`:

```
flipped = []
for c in range(n_cols):
    new_row = []
    for r in range(n_rows):
        new_row.append(scores[r][c])
    flipped.append(new_row)

# [[28, 18, 30, 21], [24, 27, 30, 19], [30, 25, 29, 26]]
```

`new_row = []` is inside the outer loop for the same reason as `total`, and for the reason on the "aliasing, one level down" slide: a fresh list each time round.

Worth noticing. Once `flipped` exists, part (b) is easy: the mean of exam `c` is `sum(flipped[c]) / len(flipped[c])`. Changing the shape of your data so that the question becomes simple is a real technique, and it is most of what data-analysis libraries do for you.

B5 *Every position*

```
xs = [3, 7, 3, 1, 3]
target = 3

positions = []
for i, x in enumerate(xs):
    if x == target:
        positions.append(i)

if len(positions) > 0:
    print(positions)
else:
    print("not found")
```

Target `3` gives `[0, 2, 4]`; `1` gives `[3]`; `8` gives `not found`.

No `break` this time: linear search stops at the first match, but here you have to see every item, because any of them might be another match. As a comprehension:

```
positions = [i for i, x in enumerate(xs) if x == target]
```

B6 *Second-largest, without sorting*

```
xs = [28, 30, 24, 18, 27]

# start from the first two, in the right order
if xs[0] >= xs[1]:
    best = xs[0]
    second = xs[1]
else:
    best = xs[1]
    second = xs[0]

# then look at the rest
for x in xs[2:]:
    if x > best:
        second = best        # the old best moves down
        best = x
    elif x > second:
        second = x           # beats second, not best

print(second)
```

input	answer	what it tests
`[28, 30, 24, 18, 27]`	`28`	the ordinary case
`[30, 24, 28]`	`28`	the `elif` branch
`[-5, -2, -9]`	`-5`	how you started
`[30, 30, 24]`	`30`	what “second” means

The two that break first attempts. `[30, 24, 28]` breaks a program without the `elif`: 28 never beats the best, so a version that only updates `second` when a new best arrives stays on 24. `[-5, -2, -9]` breaks a program that starts from `best = 0` and `second = 0`: nothing in the list beats 0, so it prints `0`, which is not in the list at all. Starting from the first two items avoids inventing a number.

`[30, 30, 24]` gives `30` with this program. Whether that is right depends on the question. "The second-highest grade in the class" is 30 — two people got it. "The second-highest *distinct* grade" is 24, and this program cannot give it with a one-line change: it starts from the first two items, and here both are 30. Neither is wrong; the specification was ambiguous, and noticing that is the answer.

Worth noticing. Sheet 4 ended with this problem and a promise that lists would make it a one-liner. They do: `sorted(xs)[-2]` gives the same answers, and is a fine way to *check* this program. The point of forbidding it is the pattern: one pass, a small number of variables that summarise everything seen so far. It is the running maximum for the fourth time, and the same shape finds the smallest, the longest, the first and the last.

B7 *A different way to sort*

pass	item moved	list after the pass	comparisons
start		`6 3 9 1 4`	
`i = 1`	`3`	`3 6 | 9 1 4`	1
`i = 2`	`9`	`3 6 9 | 1 4`	1, it stays
`i = 3`	`1`	`1 3 6 9 | 4`	3, all the way left
`i = 4`	`4`	`1 3 4 6 9`	3

b) $1 + 1 + 3 + 3 = 8$ comparisons, two fewer than selection sort.

c) On `[1, 3, 4, 6, 9]` it makes **4**, one per pass. Each new item is already bigger than the one to its left, so the `while` stops at the first comparison. Insertion sort stops looking as soon as an item is in place. Selection sort has to look at everything that is left, every pass, to be sure it has found the smallest, so it makes 10 whatever the input.

d) The reversed list `[9, 6, 4, 3, 1]`. Every item has to travel all the way to the front, and the count is $1 + 2 + 3 + 4 = 10$, the same as selection sort.

Worth noticing. So insertion sort is never worse here, and far better on a list that is almost sorted already, which real data often is. Python's own `sorted` uses this idea on short stretches of a list. Whether an algorithm's cost depends on its input is the question lecture 10 comes back to.

If you finish early

It prints `[[1, 2, 9], [3, 4]]`.

`a[:]` made a new *outer* list — but the items it copied are the two inner lists, and copying an item means copying the arrow, not the box. So after line 2, `a` and `b` are different outer lists whose arrows point at the *same* two inner lists.

- `b[0].append(9)` follows `b`'s arrow to the shared inner list and changes it. `a` sees that.
- `b[1] = [0]` changes `b`'s outer list, pointing its second slot at a brand-new list. `a`'s outer list is untouched.

A slice copies one level. This is called a *shallow* copy, and it is exactly as deep as the `[[0] * 3] * 3` problem from the lecture: the outer list is new, the rows are shared.